

Java Inheritance Laboratory Manual

Comprehensive Implementation Guide & Practical Programs (Types 1 to 5)

GLOBAL SYSTEM REQUIREMENTS

Hardware Requirements:	x86/x64 Architecture CPU (1.6 GHz or faster), 4 GB RAM, 500 MB free HDD/SSD space.
Software Requirements:	Windows 10/11, Linux, or macOS; Java Development Kit (JDK 8 or higher); Any Text Editor/IDE (VS Code, Eclipse, IntelliJ).

Program 1: Single Inheritance

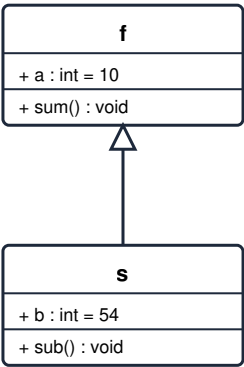
Objective: To demonstrate Single Inheritance in Java where a subclass (‘s’) inherits the state and behavior of a single superclass (‘f’).

Algorithm

1. Define a superclass named `f` with an integer field `a = 10` and a method `sum()`.
2. Define a subclass named `s` that extends class `f`, declaring its own integer variable `b = 54` and method `sub()`.
3. In the main class `Inherit1`, instantiate the subclass `s`.
4. Access superclass variable `a` and subclass variable `b` using the subclass object.
5. Invoke superclass method `sum()` and subclass method `sub()` via the subclass instance.

UML Class Diagram

UML DIAGRAM - SINGLE INHERITANCE



Source Code

```
// Single inheritance program
class f {           // super class
    int a = 10;
    void sum() {
        System.out.println("sum is " + (a + 34));
    }
}

// Sub class extending super class f
class s extends f {
    int b = 54;
    void sub() {
        System.out.println(" subtraction is " + (b - 10));
    }
}

public class Inherit1 {    // main class
    public static void main(String[] args) {
        // create an object for sub class
        s oc = new s();
        // print and call super class variable
        System.out.println(" variable can access is " + (oc.a));
        // print the sub class variable
        System.out.println(" variable can access " + (oc.b));
        // call super class and subclass methods from sub class object
        oc.sum();
        oc.sub();
    }
}
```

Compilation & Execution Steps

```
$ javac Inherit1.java $ java Inherit1
```

Program Output

```
variable can access is 10
variable can access 54
sum is 44
subtraction is 44
```

Program 2: Multilevel Inheritance

Objective: To demonstrate Multilevel Inheritance in Java where a class inherits from a child class, forming a chain of inheritance ('sea' extends 'bird', which extends 'animal').

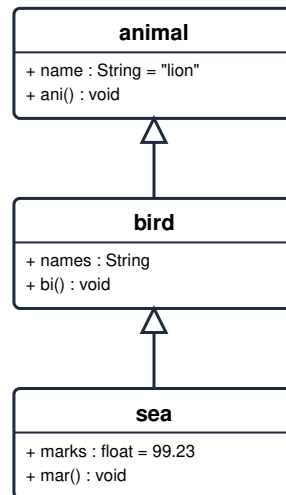
Algorithm

1. Create a base class `animal` with string variable `name` and method `ani()`.
2. Create a derived class `bird` extending `animal`, with field `names` and method `bi()`.

3. Create another derived class sea extending bird, with field marks and method mar().
4. In main class Inherit2, instantiate the lowest child class sea.
5. Invoke methods mar(), bi(), and ani() through the object of sea to prove multilevel access.

UML Class Diagram

UML DIAGRAM - MULTILEVEL INHERITANCE



Source Code

```
// Multilevel inheritance program
class animal {
    String name = " lion";
    void ani() {
        System.out.println(" lion is the my fav animal");
    }
}

class bird extends animal {
    String names = " parrot , sparrow";
    void bi() {
        System.out.println("parrot is my fav bird");
    }
}

class sea extends bird {
    float marks = 99.23f;
    void mar() {
        System.out.println(" you got 1 st rank");
    }
}

public class Inherit2 {
    public static void main(String[] args) {
        // Create an object of 3rd class (bottom of hierarchy)
        sea as = new sea();

        as.mar();
        as.bi();
    }
}
```

```

        as.ani();
    }
}

```

Compilation & Execution Steps

```
$ javac Inherit2.java $ java Inherit2
```

Program Output

```

you got 1 st rank
parrot is my fav bird
lion is the my fav animal

```

Program 3: Hierarchical Inheritance

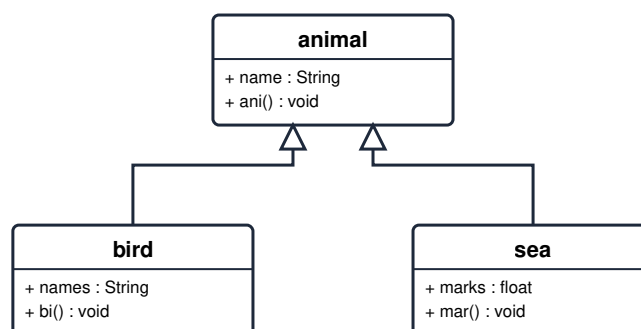
Objective: To demonstrate Hierarchical Inheritance in Java where multiple subclasses (`bird` and `sea`) inherit directly from a single parent class (`animal`).

Algorithm

1. Define a superclass `animal` containing data and method `ani()`.
2. Define subclass 1 `bird` extending `animal`.
3. Define subclass 2 `sea` extending `animal` directly.
4. In main class `Inherit3`, instantiate subclass `sea`.
5. Invoke member methods from `sea` and inherited methods from `animal`.

UML Class Diagram

UML DIAGRAM - HIERARCHICAL INHERITANCE



Source Code

```

// Hierarchical inheritance program
class animal {
    String name = " lion";
    void ani() {

```

```

        System.out.println(" lion is the my fav animal");
    }
}

class bird extends animal {
    String names = " parrot , sparrow";
    void bi() {
        System.out.println("parrot is my fav bird");
    }
}

class sea extends animal {
    float marks = 99.23f;
    void mar() {
        System.out.println(" you got 1 st rank");
    }
}

public class Inherit3 {
    public static void main(String[] args) {
        // Create an object for 3rd class (sea) inheriting from super class (animal)
        sea as = new sea();

        as.mar();
        as.ani();
        as.ani();
    }
}

```

Compilation & Execution Steps

```
$ javac Inherit3.java $ java Inherit3
```

Program Output

```

you got 1 st rank
lion is the my fav animal
lion is the my fav animal

```

Program 4: Hybrid Inheritance

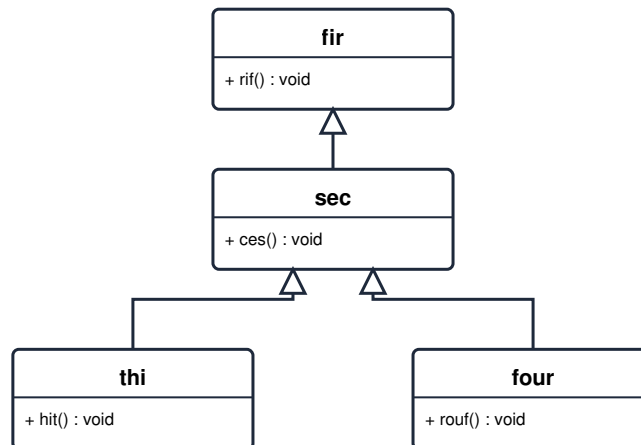
Objective: To demonstrate Hybrid Inheritance by combining Single Inheritance (`fir` → `sec`) and Hierarchical Inheritance (`sec` → `thi`, `sec` → `four`).

Algorithm

1. Define class fir with arithmetic variables and method rif().
2. Define class sec extending fir (Single Inheritance).
3. Define class thi extending sec.
4. Define class four extending sec (Hierarchical combination on sec).
5. In main class Inherit4, instantiate objects of classes four, sec, and thi to execute their respective and inherited methods.

UML Class Diagram

UML DIAGRAM - HYBRID INHERITANCE



Source Code

```
// Hybrid inheritance using Single and Hierarchical inheritance
class fir {
    int a = 10, b = 90;
    void rif() {
        System.out.println(" addition is " + (a + b));
    }
}

class sec extends fir {
    double c = 2.34f, d = 4.56f;
    void ces() {
        System.out.println(" th eresult is " + (c + d));
    }
}

class thi extends sec {
    String name = "heamnth , phk";
    void hit() {
        System.out.println(" name is" + name);
    }
}

class four extends sec {
    int f = 32, g = 34;
    void rouf() {
        System.out.println(" result is " + (f * g));
    }
}

public class Inherit4 {
    public static void main(String[] args) {
        four oj = new four();
        oj.rouf();
        oj.ces();
        oj.ces();
    }
}
```

```

        sec jo = new sec();
        jo.rif();
        jo.ces();

        thi joj = new thi();
        joj.hit();
        joj.ces();
    }
}

```

Compilation & Execution Steps

```
$ javac Inherit4.java $ java Inherit4
```

Program Output

```

result is 1088
th  eresult is 6.9000000095367432
th  eresult is 6.9000000095367432
addition is 100
th  eresult is 6.9000000095367432
name isheamnth , phk
th  eresult is 6.9000000095367432

```

Program 5: Multiple Inheritance via Interfaces

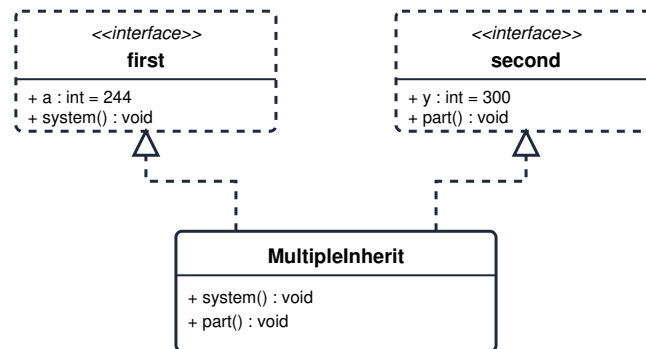
Objective: To demonstrate Multiple Inheritance in Java using interfaces (`first` and `second`) implemented by a single class (`MultipleInherit`).

Algorithm

1. Define interface `first` with constant `a = 244` and abstract method `system()`.
2. Define interface `second` with constant `y = 300` and abstract method `part()`.
3. Create class `MultipleInherit` that implements both interfaces using the `implements` keyword.
4. Provide concrete implementations for overridden methods `system()` and `part()`.
5. In `main()`, instantiate `MultipleInherit` and call the overridden interface methods.

UML Class Diagram

UML DIAGRAM - MULTIPLE INHERITANCE VIA INTERFACES



Source Code

```
// Multiple inheritance implemented using Interfaces
interface first {
    int a = 244;
    void system();
}

interface second {
    int y = 300;
    void part();
}

public class MultipleInherit implements first, second {
    @Override
    public void system() {
        System.out.println("the multiple inheritance value is " + a);
    }

    @Override
    public void part() {
        System.out.println("the second interface of value is " + y);
    }

    public static void main(String[] args) {
        MultipleInherit mi = new MultipleInherit();
        mi.system();
        mi.part();
    }
}
```

Compilation & Execution Steps

```
$ javac MultipleInherit.java $ java MultipleInherit
```

Program Output

```
the multiple inheritance value is 244
the second interface of value is 300
```